

Smart-LMS Developer Onboarding Manual

Complete System Architecture Blueprint, Data Flow Engine, API Catalog & Teammate Guide

OFFICIAL TEAMMATE GUIDE • VERSION 1.0.0 • PRODUCTION READY

TABLE OF CONTENTS

- | | |
|--|---|
| 1. Teammate Quickstart & Local Setup | 2. System Architecture & End-to-End Data Flow |
| 3. Frontend Component & State Blueprint | 4. Backend Controller & Service Reference |
| 5. Socket.IO & WebRTC Event Specification | 6. Database Schema Catalog (14 Mongoose Models) |
| 7. Complete API Endpoint Reference (21 Routes) | 8. Mathematical Algorithms & Formulas |
| 9. Deployment & Production Hardening Guide | 10. Common Pitfalls & Troubleshooting FAQ |

1. Teammate Quickstart & Local Environment Setup

1.1 Prerequisites

- Node.js:** Version 18.x or 20.x installed
- MongoDB:** Local MongoDB instance running on `mongodb://localhost:27017` OR MongoDB Atlas URI
- Package Managers:** `npm` or `pnpm`

1.2 Backend Setup (`/backend`)

```
# Step 1: Navigate to backend directory
cd backend

# Step 2: Install dependencies
npm install

# Step 3: Create .env file based on template
cp .env.example .env

# Step 4: Configure environment variables in .env
PORT=5000
MONGO_URI=mongodb://localhost:27017/smart-lms
JWT_SECRET=your_super_secret_jwt_key_here
GROQ_API_KEY=gsk_your_groq_api_key_here
GROQ_MODEL=llama-3.1-8b-instant
GEMINI_API_KEY=your_gemini_api_key_here
YOUTUBE_API_KEY=your_youtube_v3_api_key_here
CLIENT_URL=http://localhost:5173

# Step 5: Start backend server in hot-reload development mode
npm run dev
```

1.3 Frontend Setup (/frontend)

```
# Step 1: Open a new terminal and navigate to frontend directory
cd frontend

# Step 2: Install dependencies
pnpm install # or npm install

# Step 3: Create .env file
cp .env.example .env

# Step 4: Configure VITE environment variables
VITE_API_BASE_URL=http://localhost:5000/api
VITE_SOCKET_URL=http://localhost:5000

# Step 5: Start Vite development server
npm run dev
```

The application will open on <http://localhost:5173> .

2. System Architecture & End-to-End Data Flow

2.1 Complete Request-Response Lifecycle

Trace of a Student Action (e.g. Asking AI Tutor)

1. **User Input:** Student submits question in `AITutor.tsx` UI.
2. **Frontend API Client:** `src/app/lib/api.ts` intercepts request, injects `Authorization: Bearer` from `localStorage` , and dispatches POST request to `/api/ai/solve` .
3. **Security Gateway:**
 - `app.set("trust proxy", 1)` enables accurate load-balancer client IP matching.
 - `corsOptions` validates request origin against allowed domain whitelist.
 - `apiLimiter` checks rate-limit counter (prevents spam/DDoS).
 - `auth.middleware.js` decodes JWT, verifies signature, fetches user document, and sets `req.user` .
4. **Controller Layer:** `ai.controller.js` validates payload, constructs system prompt, and calls `ai.service.js` .
5. **Service & LLM Engine:** `safeGroqCall()` sends payload to Groq LLaMA 3.1 API with 30s `Promise.race()` timeout.
6. **Database & State Mutation:** Controller calculates XP earned, updates user streak, appends today's `weeklyActivity` , and updates subject mastery formula in MongoDB.
7. **Real-time Broadcast (Optional):** If social/group feature, `io.to(room).emit()` broadcasts update over Socket.IO.
8. **Client Render:** `AITutor.tsx` parses Markdown via `renderMarkdown.tsx` and renders solution.

3. Frontend Component & State Blueprint

Component Name	File Route Path	State & Features Managed
AITutor.tsx	/ai-tutor	Chat messages list, current prompt input, active tutor mode (step-by-step, hint, full solution), streaming code highlight renderer.
StudyMode.tsx & UploadMaterial.tsx	/study , /upload	File dropzone, Tesseract OCR progress spinner, extracted text viewer, AI summary card, interactive flashcard carousel.
Quiz.tsx	/quiz	Question topic selector, countdown timer, active question index, chosen answers map, final score modal with confetti animation.
Planner.tsx & AddTaskPage.tsx	/planner	Weekly schedule view, task completion checkbox triggers, risk score badge (High/Medium/Low), AI Weekly Strategy Review drawer.
Friends.tsx	/friends	Friends list, friend requests, live online status badge, direct message history, audio voice message recorder/player, WebRTC video call modal.
CriticalThinking.tsx	/critical	Brain game puzzles (Logic/Math), streak counter, XP/Level progress bar, unlocked badges grid ("Focus Beast", "Streak Monster").
YouTube.tsx	/youtube	Search bar, video cards grid, video player modal, saved videos list, watch history, AI summary generator drawer.
Analytics.tsx	/analytics	Recharts bar & line charts (questions answered, notes created, study hours), risk score gauge, readiness radar chart.

4. Backend Controller & Service Reference

4.1 Services Overview

- ai.service.js : Groq OpenAI client setup, cleanAIResponse() regex sanitizer, generateStructuredResponse() , weekly review prompt generator.
- brainAI.service.js : Logic and math brain puzzle generators, safe Groq call wrappers, game outcome generator.
- analytics.service.js : Risk calculation engine, 5-sample performance trend detector, weekly completion impact calculator, confidence engine.
- youtube.service.js : Official YouTube Data API v3 search with yt-search fallback, ISO 8601 duration regex parser, view count formatter.

5. Real-Time Socket.IO & WebRTC Event Specification Catalog

Event Name	Direction	Payload Parameters	Description
register-user	Client → Server	<code>userId: string</code>	Registers user socket in <code>onlineUsers</code> map & joins <code>user_</code> room.
online-users-list	Server → Client	<code>userIds: string[]</code>	Emits list of all currently online users to newly registered client.
user-status-change	Server → Client	<code>{ userId, online: boolean }</code>	Broadcasts online/offline status change to all connected clients.
create-group	Client → Server	<code>{ groupId, memberIds }</code>	Notifies group members and auto-joins their sockets to <code>group_</code> room.
group-message	Client → Server	<code>{ groupId, content, attachmentType, fileUrl }</code>	Saves message to DB, checks online delivery, emits to group room.
typing-group	Client → Server	<code>{ groupId, userId, userName }</code>	Broadcasts typing indicator to other group members.
call-user	Client → Server	<code>{ userToCall, signalData, from }</code>	Initiates WebRTC peer-to-peer audio/video call.
answer-call	Client → Server	<code>{ to, signalData }</code>	Completes WebRTC offer/answer handshake.

6. Complete Database Schema Catalog (14 Mongoose Models)

6.1 User Model ([src/models/user.model.js](#))

```
const userSchema = new mongoose.Schema({
  name: { type: String, trim: true },
  username: { type: String, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true },
  exam: { type: String, default: "" },
  profileImage: { type: String, default: null },
  darkMode: { type: Boolean, default: false },

  // Analytics & Metrics
  totalQuestions: { type: Number, default: 0 },
  totalNotes: { type: Number, default: 0 },
  totalQuizzes: { type: Number, default: 0 },
  totalStudyHours: { type: Number, default: 0 },
  accuracy: { type: Number, default: 0 },
  readiness: { type: Number, default: 0 },
  risk: { type: String, default: "Low" },
  weakSubject: { type: String, default: "Not Available" },
  strongSubject: { type: String, default: "Not Available" },

  // Gamification
  xp: { type: Number, default: 0 },
  level: { type: Number, default: 1 },
  rank: { type: String, default: "Beginner" },
  streak: { type: Number, default: 0 },
  lastActiveDate: { type: String, default: null },

  dailyMission: { type: dailyMissionSchema, default: null },
  weeklyActivity: { type: [weeklyActivitySchema], default: [] },
  subjectStats: { type: [subjectStatsSchema], default: [] }
}, { timestamps: true });

// Pre-save hook: auto-generates username & updates level/rank
userSchema.pre("save", function() {
  this.level = Math.floor(this.xp / 100) + 1;
  if (this.xp >= 2000) this.rank = "Grandmaster";
  else if (this.xp >= 1000) this.rank = "Master";
  else if (this.xp >= 500) this.rank = "Advanced";
  else if (this.xp >= 200) this.rank = "Intermediate";
  else this.rank = "Beginner";
});
```

7. Complete API Endpoint Reference Matrix

Subsystem	Method & Path	Auth	Request Body Payload	Response Payload Highlights
Auth	POST /api/auth/register	No	{ name, email, password, exam }	{ success: true, token: "...", user: {...} }
Auth	POST /api/auth/login	No	{ email, password }	{ success: true, token: "...", user: {...} }
AI	POST /api/ai/solve	Yes	{ question, topic }	{ success: true, answer: "# Markdown Solution..." }
Upload	POST /api/upload	Yes	file: Binary File	{ success: true, fileUrl: "http://...", fileName: "..." }
OCR	POST /api/ocr/extract	Yes	{ fileUrl }	{ success: true, text: "Extracted Plaintext..." }
MockTest	POST /api/mocktest/generate	Yes	{ subject, questionCount }	{ success: true, questions: [{ q, options, id }] }
Planner	GET /api/planner/my-plan	Yes	None	{ success: true, planner: { riskScore, weeklyPlan } }
Brain AI	POST /api/brain/submit-game	Yes	{ isCorrect, xpEarned }	{ success: true, xp, level, intelligenceScore }
Groups	POST /api/groups/create	Yes	{ name, memberIds }	{ success: true, group: { _id, name, members } }

8. Mathematical Algorithms Reference Guide

1. Risk Score Engine:

$RiskScore = clamp((100 - Accuracy) \times 0.4 + (100 - Completion) \times 0.3 + StreakImpact \times 0.3)$

2. Subject Mastery Formula:

$Mastery = \min(100, Questions \times 5 + Notes \times 7 + Quizzes \times 10)$

3. Level & Intelligence Formulas:

$Level = \lfloor XP / 100 \rfloor + 1 \quad | \quad IntelligenceScore = 100 + \lfloor XP \times 0.8 \rfloor$

9. Common Troubleshooting FAQ for Teammates

Q: Why do I get ERR_ERL_UNEXPECTED_X_FORWARDED_FOR error on server start?

This occurs when Express is behind a reverse proxy (like Render or Nginx) and `express-rate-limit` receives proxy headers without proxy trust enabled. Ensure `app.set("trust proxy", 1);` remains active in `server.js`.

Q: Why are CORS preflight OPTIONS failing from local frontend?

Make sure your local frontend URL (e.g. `http://localhost:5173`) is listed in `allowedOrigins` inside `server.js` or added to `CLIENT_URL` in your `.env` file.

Q: Tesseract OCR is taking long or failing to initialize?

Ensure `eng.traineddata` is present in the root/backend folder or accessible via internet so Tesseract.js can load the English OCR language model worker thread.

